GURU GHASIDAS VISHWAVIDYALA

BILASPUR,C.G

Lab Record

B.Sc in Computer Science(4th Semester)

NAME: Rahul Hansda.

ROLL.NO: 24070156.

ENROLLMENT.NO: GGV/24/05156.

COURSE: Object Oriented Programming with java.

YEAR: 2026.

Department of Computer Science s Information Technology.

```
import java.util.*;
import java.io.*;

public class P1 {
    // We create a helper class inside to keep the data linked
    // This ensures that name, city, and quantity stay
together during sorting
    class FarmerRecord {
        String name, fruit, city;
        int qty;

        FarmerRecord(String n, String f, String c, int q) {
            this.name = n; this.fruit = f; this.city = c; this.qty =
q;
        }

        @Override
        public String toString() {
            return String.format("%-15s %-12s %-12s %-8d", name,
fruit, city, qty);
        }
    }

    FarmerRecord[] records = new FarmerRecord[10];

    void input() {
        Scanner in = new Scanner(System.in);
        for (int i = 0; i < 10; i++) {
            System.out.println("\nFarmer #" + (i + 1));
            System.out.print("Enter Name: ");
            String n = in.nextLine();
            System.out.print("Enter Fruit: ");
            String f = in.nextLine();
            System.out.print("Enter City: ");
            String c = in.nextLine();
            System.out.print("Enter Quantity: ");
            int q = Integer.parseInt(in.nextLine());

            records[i] = new FarmerRecord(n, f, c, q);
        }
```

```
    }

    void saveAndDisplay(String title, String filename, boolean
append) {
        try (PrintWriter writer = new PrintWriter(new
FileWriter(filename, append))) {
            System.out.println("\n--- " + title + " ---");
            writer.println("\n" + title);
            writer.println("-----------------------------------------------
--------------");
            writer.printf("%-15s %-12s %-12s %-8s\n", "NAME",
"FRUIT", "CITY", "QTY");

            for (FarmerRecord r : records) {
                System.out.println(r);
                writer.println(r);
            }
            writer.close();
        } catch (IOException e) {
            System.out.println("File Error!");
        }
    }

    void sortData() {
        // Using built-in Arrays.sort with a custom rule
(Comparator)
        // This sorts the 'records' based on the 'qty' field
        Arrays.sort(records, (a, b) -> Integer.compare(a.qty,
b.qty));
        System.out.println("\n[System] Data sorted by quantity
successfully.");
    }

    void search(Scanner in) {
        System.out.print("\nEnter farmer name to search: ");
        String target = in.nextLine();
        boolean found = false;
        for (FarmerRecord r : records) {
            if (r.name.equalsIgnoreCase(target)) {
                System.out.println("Record Found: " + r);
```

```
                found = true;
                break;
            }
        }
        if (!found) System.out.println("No farmer found with
that name.");
    }

    public static void main(String args[]) {
        P1 obj = new P1();
        Scanner sc = new Scanner(System.in);
        String file = "farmers_notepad.txt";

        obj.input();

        // Task 1: Save and show before sorting
        obj.saveAndDisplay("DATA BEFORE SORTING", file,
false);

        // Task 2: Sort
        obj.sortData();

        // Task 3: Save and show after sorting
        obj.saveAndDisplay("DATA AFTER SORTING", file, true);

        // Task 4: Search
        obj.search(sc);
    }
}
```

OUTPUT

```
DATA BEFORE SORTING
-------------------------------------------------------
NAME          FRUIT        CITY         QTY
a             apple        bilaspur     23
b             guava        koni         45
c             watermelon   birkona      35
d             grapes       raipur       36
e             papaya       sakti        65
```

f	jackfruit	raigarh	76
g	custardapple	sarkanda	65
h	coconut	sipat	23
i	lichy	dhanbad	78
j	banana	raiganj	78

DATA AFTER SORTING

NAME	FRUIT	CITY	QTY
a	apple	bilaspur	23
h	coconut	sipat	23
c	watermelon	birkona	35
d	grapes	raipur	36
b	guava	koni	45
e	papaya	sakti	65
g	custardapple	sarkanda	65
f	jackfruit	raigarh	76
i	lichy	dhanbad	78
j	banana	raiganj	78

Enter farmer name to search: j
Record Found: j banana raiganj 78

```
import java.io.IOException;

public class P2 {
    public static void main(String[] args) throws IOException {
        // 1. Data Storage: Parallel Arrays (No inbuilt
Collections)
        String[] fruitNames = {"Mango", "Apple", "Papaya"};
        double[] quantities = new double[3];
        String[] qualities = new String[3];

        System.out.println("--- Enter Fruit Details ---");

        for (int i = 0; i < 3; i++) {
            System.out.print("Enter quantity for " +
fruitNames[i] + ": ");
            String qtyStr = readLine();
            double qty = parseDouble(qtyStr);

            System.out.print("Enter quality for " + fruitNames[i]
+ ": ");
            String qual = readLine();

            // Logic: Store half if > 1000
            if (qty > 1000) {
                qty = qty / 2;
                System.out.println("  > Storing half: " + qty);
            }

            quantities[i] = qty;
            qualities[i] = qual;
        }

        // 2. Manual Search Logic
        System.out.println("\n--- Search Inventory ---");
        System.out.print("Enter fruit name: ");
        String searchName = readLine();
        System.out.print("Enter quality: ");
        String searchQual = readLine();

        boolean found = false;
```

```
        for (int i = 0; i < 3; i++) {
            // Manual comparison without complex library
functions
            if (fruitNames[i].equalsIgnoreCase(searchName) &&
qualities[i].equalsIgnoreCase(searchQual)) {
                System.out.println("\nMatch Found!");
                System.out.println("Fruit: " + fruitNames[i]);
                System.out.println("Stored Qty: " + quantities[i]);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("\nNo match found for that
combination.");
        }
    }

    // Manual Input Handling: Reads characters byte-by-byte
until Enter is pressed
    public static String readLine() throws IOException {
        StringBuilder sb = new StringBuilder();
        int character;
        while ((character = System.in.read()) != '\n' &&
character != -1) {
            if (character != '\r') { // Ignore carriage return for
Windows compatibility
                sb.append((char) character);
            }
        }
        return sb.toString().trim();
    }

    // Manual Conversion: String to Double logic
    public static double parseDouble(String s) {
        try {
            return Double.parseDouble(s);
        } catch (NumberFormatException e) {
            return 0.0;
```

```
        }
    }
}
```

OUTPUT

```
--- Enter Fruit Details ---
Enter quantity for Mango: 2314
Enter quality for Mango: Good
  > Storing half: 1157.0
Enter quantity for Apple: 231
Enter quality for Apple: Average
Enter quantity for Papaya: 4523
Enter quality for Papaya: Bad
  > Storing half: 2261.5

--- Search Inventory ---
Enter fruit name: Papaya
Enter quality: Bad

Match Found!
Fruit: Papaya
Stored Qty: 2261.5
```

```
import java.util.Scanner;

class Orchard {
    String id;
    String fruit;
    String quality;

    Orchard(String id, String fruit, String quality) {
        this.id = id;
        this.fruit = fruit;
        this.quality = quality;
    }
}

public class P3 {
    public static void main(String[] args) {
        // Data Initialization based on problem statement
        Orchard[] orchards = {
            new Orchard("A", "Apple", "Good"),
            new Orchard("B", "Litchi", "Good"),
            new Orchard("C", "Mango", "Good"),
            new Orchard("D", "Apple", "Bad")
        };

        // 1. Display the initial Tree View
        printTreeView("Root_Farm", orchards, "");

        // 2. Search Operation
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit name to search: ");
        String searchFruit = sc.nextLine().trim();

        System.out.println("\n--- Search Results ---");
        boolean found = false;

        // Custom tree-view for search results
        System.out.println("Search_Results");
        for (Orchard o : orchards) {
            if (o.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println(" └─── Orchard " + o.id + " ["
```

```
+ o.fruit + " | Quality: " + o.quality + "]");
                found = true;
            }
        }

        if (!found) {
            System.out.println(" └─── No orchards found
containing: " + searchFruit);
        }

        sc.close();
    }

    /**
     * Helper method to print a structured tree view
     */
    public static void printTreeView(String rootName,
Orchard[] orchards, String filter) {
        System.out.println(rootName);
        for (int i = 0; i < orchards.length; i++) {
            // Use different branch symbols for the last item
            String branch = (i == orchards.length - 1) ? " └─── "
: " ├─── ";

            System.out.println(branch + "Orchard " +
orchards[i].id);
            System.out.println((i == orchards.length - 1 ? "    " : "
│   ") + " ├─── Fruit: " + orchards[i].fruit);
            System.out.println((i == orchards.length - 1 ? "    " : "
│   ") + " └─── Quality: " + orchards[i].quality);
        }
    }
}
```

OUTPUT

```
Root_Farm
├── Orchard A
│   ├── Fruit: Apple
│   └── Quality: Good
├── Orchard B
│   ├── Fruit: Litchi
│   └── Quality: Good
├── Orchard C
│   ├── Fruit: Mango
│   └── Quality: Good
└── Orchard D
    ├── Fruit: Apple
    └── Quality: Bad

Enter fruit name to search: Apple

--- Search Results ---
Search_Results
└── Orchard A [Apple | Quality: Good]
└── Orchard D [Apple | Quality: Bad]
```

```
import java.util.Scanner;

public class P4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // 1. Setup the Map (Graph)
        // Let's assume 6 points (0 to 5) for this example
        int totalNodes = 6;
        int[][] adjMatrix = new int[totalNodes][totalNodes];
        boolean[] visited = new boolean[totalNodes];

        // Manually providing edges (The "Map")
        // Component 1: 0-1, 1-2
        addEdge(adjMatrix, 0, 1);
        addEdge(adjMatrix, 1, 2);

        // Component 2: 3-4
        addEdge(adjMatrix, 3, 4);

        // Component 3: 5 (Isolated point)

        System.out.println("--- Graph Subtrees (Connected
Components) ---");
        int subtreeCount = 0;

        for (int i = 0; i < totalNodes; i++) {
            if (!visited[i]) {
                subtreeCount++;
                System.out.println("\nSubtree " + subtreeCount +
":");
                displaySubtreeDFS(i, adjMatrix, visited, "Root");
            }
        }
    }

    // Manual method to add an edge to the Adjacency Matrix
    public static void addEdge(int[][] matrix, int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1; // Undirected graph
```

```
    }

    /**
     * Depth First Search (DFS) to find and display connected
nodes
     * This mimics the "Tree View" from the previous question.
     */
    public static void displaySubtreeDFS(int node, int[][]
matrix, boolean[] visited, String prefix) {
        visited[node] = true;
        System.out.println(prefix + "── Point " + node);

        for (int neighbor = 0; neighbor < matrix.length;
neighbor++) {
            // If there is a connection and the neighbor hasn't
been visited
            if (matrix[node][neighbor] == 1 &&
!visited[neighbor]) {
                // Adjust prefix for visual tree representation
                displaySubtreeDFS(neighbor, matrix, visited, "
└────");
            }
        }
    }
}
```

OUTPUT

```
--- Graph Subtrees (Connected Components) ---

Subtree 1:
Root── Point 0
    └──── Point 1
    └──── Point 2

Subtree 2:
Root── Point 3
    └──── Point 4

Subtree 3:
Root── Point 5
```

```
import java.util.Scanner;

public class P5 {
    // Adjacency Matrix for the graph (7 nodes)
    static int[][] adjMatrix = new int[7][7];
    static boolean[] visited = new boolean[7];
    static int[] path = new int[7]; // To store the successful
path

    public static void main(String[] args) {
        // Initialize the Map (Graph) with edges
        addEdge(0, 1); addEdge(0, 2);
        addEdge(1, 3); addEdge(1, 4);
        addEdge(2, 5); addEdge(4, 6);

        Scanner sc = new Scanner(System.in);
        System.out.println("--- Graph Navigation ---");
        System.out.print("Enter Entry Point (0-6): ");
        int start = sc.nextInt();
        System.out.print("Enter Goal Point (0-6): ");
        int goal = sc.nextInt();

        System.out.println("\nSearching for matching
subtree...");

        if (findPath(start, goal, 0)) {
            System.out.println("\nMatching Subtree Found:");
            displayPathAsTree(0);
        } else {
            System.out.println("\nNo path exists between " +
start + " and " + goal);
        }
    }

    static void addEdge(int u, int v) {
        adjMatrix[u][v] = 1;
        adjMatrix[v][u] = 1;
    }

    /**
```

```
     * Recursive DFS to find the path to the goal
     */
    static boolean findPath(int current, int goal, int depth) {
        visited[current] = true;
        path[depth] = current; // Add current node to path

        if (current == goal) {
            // We found the goal; mark the end of path with a
sentinel value
            if (depth + 1 < path.length) path[depth + 1] = -1;
            return true;
        }

        for (int i = 0; i < adjMatrix.length; i++) {
            if (adjMatrix[current][i] == 1 && !visited[i]) {
                if (findPath(i, goal, depth + 1)) {
                    return true;
                }
            }
        }

        // Backtrack: If no neighbor leads to goal, this node
isn't part of the path
        visited[current] = false;
        return false;
    }

    /**
     * Prints the stored path in a tree-view format
     */
    static void displayPathAsTree(int depth) {
        if (path[depth] == -1) return;

        String indent = "";
        for (int i = 0; i < depth; i++) {
            indent += "    ";
        }

        String branch = (depth == 0) ? "Root: " : " └─── ";
        System.out.println(indent + branch + "Point " +
```

```
path[depth]);

        // Stop if we reached the end of the stored path
        if (depth + 1 < path.length && path[depth + 1] != -1) {
            displayPathAsTree(depth + 1);
        }
    }
}
```

OUTPUT

```
--- Graph Navigation ---
Enter Entry Point (0-6): 4
Enter Goal Point (0-6): 2

Searching for matching subtree...

Matching Subtree Found:
Root: Point 4
    └── Point 1
        └── Point 0
            └── Point 2
```

```
import java.util.Scanner;

public class P6 {
    static final int INF = Integer.MAX_VALUE;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // 1. Define the weighted graph (6 nodes: 0 to 5)
        // 0 represents no connection
        int[][] graph = {
            {0, 4, 2, 0, 0, 0}, // Node 0
            {4, 0, 1, 5, 0, 0}, // Node 1
            {2, 1, 0, 8, 10, 0},// Node 2
            {0, 5, 8, 0, 2, 6}, // Node 3
            {0, 0, 10, 2, 0, 3},// Node 4
            {0, 0, 0, 6, 3, 0}  // Node 5
        };

        System.out.println("--- Shortest Path Finder (Dijkstra) -
--");
        System.out.print("Enter Start Node (0-5): ");
        int start = sc.nextInt();
        System.out.print("Enter End Node (0-5): ");
        int end = sc.nextInt();

        findShortestPath(graph, start, end);
    }

    public static void findShortestPath(int[][] graph, int start,
int end) {
        int n = graph.length;
        int[] dist = new int[n];      // Shortest distance from
start to node i
        boolean[] visited = new boolean[n]; // True if node i is
processed
        int[] parent = new int[n];    // To reconstruct the path

        // Initialize distances as Infinity and parent as -1
        for (int i = 0; i < n; i++) {
```

```
            dist[i] = INF;
            visited[i] = false;
            parent[i] = -1;
        }

        dist[start] = 0;

        for (int i = 0; i < n - 1; i++) {
            // Pick the minimum distance node from the set of
unvisited nodes
            int u = minDistance(dist, visited);
            if (u == -1) break; // All reachable nodes processed

            visited[u] = true;

            // Update distance values of the adjacent nodes
            for (int v = 0; v < n; v++) {
                if (!visited[v] && graph[u][v] != 0 && dist[u] !=
INF
                    && dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                    parent[v] = u;
                }
            }
        }

        printResult(start, end, dist[end], parent);
    }

    // Helper to find the node with the smallest distance
    public static int minDistance(int[] dist, boolean[] visited)
{
        int min = INF, minIndex = -1;
        for (int v = 0; v < dist.length; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                minIndex = v;
            }
        }
        return minIndex;
```

```
    }

    public static void printResult(int start, int end, int
totalDist, int[] parent) {
        if (totalDist == INF) {
            System.out.println("\nNo path exists between node "
+ start + " and " + end);
            return;
        }

        System.out.println("\nShortest Distance: " + totalDist);
        System.out.print("Path: ");
        printPath(parent, end);
        System.out.println();
    }

    // Recursive function to print the path from parent array
    public static void printPath(int[] parent, int j) {
        if (parent[j] == -1) {
            System.out.print(j);
            return;
        }
        printPath(parent, parent[j]);
        System.out.print(" -> " + j);
    }
}
```

OUTPUT

```
--- Shortest Path Finder (Dijkstra) ---
Enter Start Node (0-5): 4
Enter End Node (0-5): 4


Shortest Distance: 0
Path: 4
```

```
import java.util.Scanner;

public class P7 {

    // 1. 'throws' keyword: Declares that this method might
throw an exception
    // It warns the caller to handle the potential error.
    public static void validateAge(int age) throws
ArithmeticException {
        if (age < 18) {
            // 2. 'throw' keyword: Used to explicitly throw a
specific exception
            throw new ArithmeticException("Access denied - You
must be at least 18 years old.");
        } else {
            System.out.println("Access granted - You are old
enough!");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your age: ");

        // 3. 'try' keyword: Wraps the code that might cause an
error
        try {
            int ageInput = sc.nextInt();
            validateAge(ageInput);

            // This line won't execute if an exception occurs
above
            int result = 10 / 0;

        }
        // 4. 'catch' keyword: Handles the exception if one is
thrown in the try block
        catch (ArithmeticException e) {
            System.out.println("Caught an Exception: " +
e.getMessage());
```

```
        }
        catch (Exception e) {
            System.out.println("Caught a general error: " + e);
        }
        // 5. 'finally' keyword: Code that ALWAYS runs,
regardless of an error
        finally {
            System.out.println("\n--- Cleanup Operations ---");
            System.out.println("Closing scanner and finalizing
process.");
            sc.close();
        }

        System.out.println("Program continues execution after
the blocks...");
    }
}
```

OUTPUT

```
Enter your age: 21
Access granted - You are old enough!
Caught an Exception: / by zero

--- Cleanup Operations ---
Closing scanner and finalizing process.
Program continues execution after the blocks...
```

```
import java.util.Scanner;

// 1. Define the Custom Exception
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    // 2. Method that 'throws' the custom exception
    public void withdraw(double amount) throws
NoMoneyException {
        System.out.println("\nAttempting to withdraw: $" +
amount);

        if (amount > balance) {
            // 3. 'throw' the exception if logic fails
            throw new NoMoneyException("Insufficient Funds!
Your balance is only: $" + balance);
        }

        balance -= amount;
        System.out.println("Withdrawal successful! Remaining
balance: $" + balance);
    }

    public double getBalance() {
        return balance;
    }
}

public class P8 {
    public static void main(String[] args) {
```

```
        BankAccount myAccount = new BankAccount(1000.00);
// Initial balance $1000
        Scanner sc = new Scanner(System.in);

        System.out.println("Current Balance: $1000.00");
        System.out.print("Enter amount to withdraw: ");

        try {
            double amount = sc.nextDouble();
            myAccount.withdraw(amount);
        }
        catch (NoMoneyException e) {
            // Handling our specific custom exception
            System.err.println("Transaction Failed: " +
e.getMessage());
        }
        catch (Exception e) {
            System.err.println("An unexpected error occurred.");
        }
        finally {
            System.out.println("Final Account Balance: $" +
myAccount.getBalance());
            sc.close();
        }
    }
}
```

OUTPUT

```
Current Balance: $1000.00
Enter amount to withdraw: 500

Attempting to withdraw: $500.0
Withdrawal successful! Remaining balance: $500.0
Final Account Balance: $500.0
```

```
class Platform {
    // The 'synchronized' keyword ensures only one thread
executes this at a time
    public synchronized void usePlatform(String trainName) {
        System.out.println("\n[SIGNAL] Platform is now CLEAR
for " + trainName);
        System.out.println(trainName + " is entering the
platform...");

        try {
            // Simulating the time the train stays at the
platform
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(trainName + " has departed the
platform.");
        System.out.println("[SIGNAL] Platform is now
VACANT.");
    }
}

class Train extends Thread {
    String name;
    Platform platform;

    Train(String name, Platform platform) {
        this.name = name;
        this.platform = platform;
    }

    @Override
    public void run() {
        // Each train attempts to use the same platform
object
        platform.usePlatform(name);
    }
}
```

```
public class P9 {
    public static void main(String[] args) {
        Platform sharedPlatform = new Platform();

        // Creating multiple train threads
        Train t1 = new Train("Express 101", sharedPlatform);
        Train t2 = new Train("Freight 202", sharedPlatform);
        Train t3 = new Train("Passenger 303", sharedPlatform);

        // Starting the threads
        // Even if they start at the same time, synchronization
forces them to queue
        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT

```
[SIGNAL] Platform is now CLEAR for Express 101
Express 101 is entering the platform...
Express 101 has departed the platform.
[SIGNAL] Platform is now VACANT.

[SIGNAL] Platform is now CLEAR for Passenger 303
Passenger 303 is entering the platform...
Passenger 303 has departed the platform.
[SIGNAL] Platform is now VACANT.

[SIGNAL] Platform is now CLEAR for Freight 202
Freight 202 is entering the platform...
Freight 202 has departed the platform.
[SIGNAL] Platform is now VACANT.
```

```
class Kitchen {
    private boolean isKitchenFree = true;
    private String currentOrder = "";

    // Waiter calls this to place an order
    public synchronized void acceptOrder(String order) {
        // If kitchen is busy, waiter must wait
        while (!isKitchenFree) {
            try {
                System.out.println("[Waiter] Kitchen is busy.
Waiting with order: " + order);
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        // Place the order
        this.currentOrder = order;
        isKitchenFree = false;
        System.out.println("[Waiter] Placed order: " + order);

        // Notify the Chef that an order is ready to be cooked
        notify();
    }

    // Chef calls this to deliver the order
    public synchronized void deliverOrder() {
        // If kitchen is free, there is nothing to cook; Chef
waits
        while (isKitchenFree) {
            try {
                System.out.println("[Chef] No orders. Chef is
resting...");
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
```

```
        // Cook and deliver
        System.out.println("[Chef] Cooking and delivering: " +
currentOrder);
        try { Thread.sleep(1500); } catch (InterruptedException
e) {} // Simulating cooking

        isKitchenFree = true;
        currentOrder = "";
        System.out.println("[Chef] Order delivered! Kitchen is
now free.");

        // Notify the Waiter that kitchen is free for the next
order
        notify();
    }
}

class Waiter extends Thread {
    Kitchen kitchen;
    String[] orders = {"Pasta", "Pizza", "Burger"};

    Waiter(Kitchen kitchen) { this.kitchen = kitchen; }

    public void run() {
        for (String item : orders) {
            kitchen.acceptOrder(item);
        }
    }
}

class Chef extends Thread {
    Kitchen kitchen;

    Chef(Kitchen kitchen) { this.kitchen = kitchen; }

    public void run() {
        // Chef works in a loop to handle incoming orders
        for (int i = 0; i < 3; i++) {
            kitchen.deliverOrder();
        }
```

```
    }
}

public class P10 {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();

        Waiter waiter = new Waiter(kitchen);
        Chef chef = new Chef(kitchen);

        // Start both threads
        chef.start();   // Chef starts looking for orders
        waiter.start(); // Waiter starts taking orders
    }
}
```

OUTPUT

```
[Waiter] Placed order: Pasta
[Waiter] Kitchen is busy. Waiting with order: Pizza
[Chef] Cooking and delivering: Pasta
[Chef] Order delivered! Kitchen is now free.
[Waiter] Placed order: Pizza
[Waiter] Kitchen is busy. Waiting with order: Burger
[Chef] Cooking and delivering: Pizza
[Chef] Order delivered! Kitchen is now free.
[Waiter] Placed order: Burger
[Chef] Cooking and delivering: Burger
[Chef] Order delivered! Kitchen is now free.
```